OFF CAMERA – Der vollständige Entwicklungsleitfaden

Projekt: OFF CAMERA
Ziel: Eine spielbare Singleplayer-Lebens- und Managementsimulation, die nativ auf dem eigenen iPhone läuft.
Plattform: iOS zuerst und ausschließlich; kein Android-Ziel und keine App-Store-Veröffentlichung geplant.
Technik: Godot 4.x + GDScript + Claude Code + Git/GitHub + macOS/Xcode.
Budget-Leitplanke: maximal 100 US-Dollar für KI-/Modellnutzung; Softwarekosten und eventuelle Apple-Kosten separat prüfen.
Dokumentstatus: Umsetzungsplan, Arbeitsanweisung und kopierbarer Master-Prompt.

Inhaltsverzeichnis

1. Ziel und Spielvision
2. Was „spielbar“ konkret bedeutet
3. Technische Entscheidungen
4. Rollen: Du, Claude Code und die Tools
5. Einmalige Einrichtung auf dem Mac
6. GitHub-Repository einrichten
7. Godot-Projekt anlegen und iOS konfigurieren
8. Projektordner und Architektur
9. Die nicht verhandelbaren Kernsysteme
10. Entwicklungsphasen mit konkreten Abnahmekriterien
11. iPhone-Build, Installation und Tests
12. Speicherstände, Fehlerbehebung und Qualität
13. Kosten- und Tokenkontrolle
14. Arbeitsablauf mit Claude Code

15. Master-Prompt zum Kopieren
16. Folgeprompts für jede Entwicklungsphase
17. Checklisten und Definition of Done
18. Häufige Probleme
19. Die ersten zwei Arbeitssitzungen

1. Ziel und Spielvision

1.1 Das Spiel in einem Absatz

OFF CAMERA ist eine mobile Singleplayer-Lebens- und Managementsimulation über eine Person, die in Marbella lebt, eine öffentliche Karriere als Streamer/Creator aufbaut und parallel ein wachsendes, fiktionales Unterwelt-Unternehmen managt. Das Spiel verbindet Alltag, Beziehungen, Termine, Streaming, Einnahmen, Ausgaben, Personal, Produktionsstandorte, internationale Expansion, Risiko und den Druck durch Behörden. Der Spieler soll über gut strukturierte Menüs planen und entscheiden können und zugleich eine Stadtkarte nutzen, um Orte zu entdecken und Aktivitäten auszuwählen.

Der Reiz entsteht aus dem Zusammenspiel der Systeme: Ein Streaming-Termin belegt Zeit, verbessert möglicherweise Reichweite und Einnahmen, kostet Energie und kann mit einem privaten Termin kollidieren. Ein neuer Standort erzeugt laufende Kosten, braucht Personal und verändert Gewinn sowie Risiko. Größere und auffällige Geldströme können Prüfungen und Ereignisse auslösen. Alle Systeme müssen dieselbe Zeit- und Finanzlogik verwenden.

1.2 Spielgefühl und Designprinzipien

- **Mobile-first:** Das Spiel ist für Touch-Bedienung und ein iPhone-

Display entworfen, nicht für Maus und Tastatur verkleinert.
- **Singleplayer:** Keine Serverpflicht, kein Multiplayer und keine Onlinekonten als Voraussetzung für den Spielstand.
- **Management mit direkter Übersicht:** Komplexität liegt in den Systemen, die Bedienung bleibt verständlich.
- **Spielerische Entscheidungen:** Jede Aktivität sollte einen erkennbaren Preis, Nutzen, Zeitaufwand oder eine Konsequenz haben.
- **Ein gemeinsamer Zeitfluss:** Es darf keine separaten Uhren für Streaming, Alltag, Firma oder Behörden geben.
- **Eine nachvollziehbare Finanzhistorie:** Jede relevante Veränderung von Geld hat eine Quelle und einen Grund.
- **Skalierbare Inhalte:** Neue Städte, Länder, Firmen und Ereignisse sollen möglichst über Daten ergänzt werden, nicht durch grundlegende Code-Umbauten.
- **Iterativ:** Erst einen kleinen funktionierenden Kern bauen, dann Systeme erweitern.
- **Keine Zeitbeschleunigung als Grundmechanik:** Zeit vergeht über klar definierte Aktionen und Ereignisse. Keine unkontrollierte Echtzeit-Simulation im Hintergrund.
- **Fiktionale Welt:** Namen, Organisationen, Figuren und Ereignisse sollen als eigenständige Fiktion gestaltet werden. Bekannte Personen können höchstens lose Inspirationsquelle sein; keine Behauptungen über reale Personen.

1.3 Geplante Spielbereiche

A. Alltag und Privatleben

- Tages- und Wochenkalender
- Energie, Stimmung, Gesundheit als abstrakte Spielwerte und verfügbare Zeit
- Schlafen, Essen, Freizeit, Training, Einkaufen und soziale Termine
- Beziehungen zu ausgewählten Figuren
- Konflikte zwischen Arbeit, Freizeit und Verpflichtungen

- Orte auf der Marbella-Karte mit jeweils passenden Aktivitäten

B. Streamer-/Creator-Karriere

- Kanalprofil und Reichweite
- Content-Formate und Streaming-Aktivitäten
- Zuschauerentwicklung und Community-Stimmung
- Einnahmen aus abstrakten Quellen wie Werbepartnerschaften, Mitgliedschaften und Kooperationen
- Equipment und Upgrades
- Termine, Energieverbrauch, Reputation und Zeitkonflikte
- Ereignisse wie virale Clips, technische Probleme oder Kooperationen

C. Fiktionales Unternehmen / Unterwelt-Management

- Unternehmensübersicht, Standorte und Abteilungen
- Personal, laufende Kosten, Kapazität, Produktivität und abstrakte Risiken
- Ausbau und Eröffnung weiterer Standorte
- Managemententscheidungen mit Zeit- und Geldkosten
- Ereignisse, die die Stabilität oder den Druck auf die Organisation beeinflussen
- Keine realweltlich umsetzbaren Anleitungen zu Straftaten; Aktivitäten werden abstrahiert als Spielwerte und Entscheidungen dargestellt.

D. Finanzen, Buchführung und Behörden

- Zentrales Konto- und Buchungsjournal
- Einnahmen, Ausgaben, laufende Kosten und Vermögensübersicht
- Steuer- und Compliance-Ereignisse als Spielmechanik
- Prüfungen, Nachfragen, Fristen, Bußgelder oder rechtliche Konsequenzen als fiktionale Ereignisketten
- Konsequenzen entstehen aus nachvollziehbaren Spielregeln, nicht aus willkürlichen Zufallsstrafen
- Keine Anleitung zu Steuerhinterziehung, Geldwäsche oder dem

Umgehen von Ermittlungen; das Spiel modelliert Risiken abstrakt und kann legale Optionen, Beratung, Dokumentation und Kooperation als Entscheidungen anbieten.

E. Stadt, Standorte und Expansion

- Interaktive, vereinfachte Stadtkarte
- Orte wie Wohnung, Studio, Fitnessstudio, Café, Büro und weitere Spielorte
- Orte zeigen Öffnungszeiten oder Verfügbarkeit nur, wenn diese Regeln tatsächlich implementiert sind
- Detailverwaltung über gut lesbare Menüs
- Spätere Erweiterung um weitere Städte und Länder

---

2. Was „spielbar“ konkret bedeutet

Ein Projekt ist nicht spielbar, nur weil Godot ein Fenster öffnet oder Claude Code viele Dateien erstellt hat. Der erste Meilenstein gilt erst dann als erreicht, wenn du auf deinem iPhone Folgendes zuverlässig machen kannst:

1. App starten und das Hauptmenü sehen.
2. Ein neues Spiel beginnen.
3. Einen Spieltag mit einem klaren Datum und Kontostand sehen.
4. Eine Aktivität auswählen, beispielsweise „Streamen“ oder „Ausruhen“.
5. Die Aktivität starten und eine definierte Menge Spielzeit verbrauchen.
6. Einen nachvollziehbaren Effekt sehen, beispielsweise mehr Reichweite, weniger Energie oder eine Ausgabe.
7. Eine Buchung im Finanzjournal finden, falls Geld verändert wurde.
8. Ein Ereignis erhalten, wenn eine definierte Bedingung eintritt.

9. Den Spielstand speichern.
10. Die App vollständig schließen und wieder öffnen.
11. Den gespeicherten Spielstand laden und dieselben Werte wiederfinden.
12. Die wichtigsten Buttons mit dem Finger bedienen können, ohne dass Inhalte abgeschnitten werden.
13. Ohne Netzwerkverbindung weiterspielen können, sofern keine ausdrücklich optionale Funktion Internet braucht.

Regel: Nach jeder Entwicklungsphase muss ein benutzbarer, testbarer Stand existieren. Große Systeme dürfen nicht auf Kosten eines funktionierenden Spiels vorgezogen werden.

---

3. Technische Entscheidungen

3.1 Empfohlener Stack

Aufgabe	Entscheidung	Warum
Game Engine	Godot 4.x, stabile Version	Kostenlos, für 2D-Managementspiele gut geeignet, iOS-Export möglich
Programmiersprache	GDScript	Passt nativ zu Godot; für Claude Code gut lesbar und schnell iterierbar
Zielplattform	iOS auf iPhone	Kein paralleles Android-Ziel, weniger Plattformumfang

Editor für das Spiel	Godot Editor	Szenen, Nodes, UI und Spiel starten/testen
Coding-Assistent	Claude Code	Kann Projektdateien lesen, ändern, Tests und Dokumentation erstellen
Versionsverwaltung	Git	Änderungen nachverfolgen, Fehler zurückrollen, Meilensteine sichern
Remote-Repository	GitHub Private Repository	Backup und zentraler Projektstand
iOS-Build	Godot iOS Export + Xcode auf Mac	Native iOS-App signieren und auf Gerät testen
Datenformat	Godot Resources und/oder JSON für Inhalte; Save-Datei lokal	Daten und Logik trennen; Speichern kontrollierbar halten
Teststrategie	Godot-Tests/kleine Debug-Szenen + manuelle iPhone-Tests	Kernregeln müssen reproduzierbar sein

Versionsregel: Installiere eine stabile Godot-4-Version und notiere die genaue Versionsnummer im Projekt-README. Aktualisiere nicht mitten in einer kritischen Phase ohne Grund. Die konkrete stabile Version und die aktuellen iOS-Export-Anforderungen bitte bei Beginn auf den offiziellen Seiten prüfen.

3.2 iOS-Realität

- Für den üblichen nativen iOS-Export aus Godot brauchst du einen Mac mit macOS und Xcode.

- Du musst das Spiel nicht im App Store veröffentlichen, um auf einem eigenen iPhone zu testen.
- Für die Installation auf einem Gerät brauchst du trotzdem Signierung. Ein kostenloses Apple-Konto kann für persönliche Tests ausreichen, unterliegt aber Einschränkungen und kann regelmäßige Neuinstallation/Neusignierung erforderlich machen.
- Eine bezahlte Apple-Developer-Mitgliedschaft ist vor allem für bestimmte längere Test- und Verteilungsabläufe relevant. Prüfe Apples aktuelle Regeln, bevor du Geld ausgibst.
- Für iOS-Export kann Godot ein Xcode-Projekt erzeugen; Xcode übernimmt den gerätespezifischen Build und die Signierung.
- Ein iPhone-Simulator ist nützlich, ersetzt aber Tests auf dem echten iPhone nicht.
- App-Berechtigungen so sparsam wie möglich halten. Für den ersten Prototyp sind Kamera, Mikrofon, Kontakte, Standort und Push-Mitteilungen nicht nötig.
- Offline-Speichern ist für die erste Version Pflicht. Cloud-Sync wird nicht vorausgesetzt.

3.3 Was GitHub leistet – und was nicht

GitHub speichert Quellcode, Dokumentation und Versionshistorie. Claude Code kann Änderungen am lokalen Repository vornehmen, und du kannst diese mit Git committen und nach GitHub pushen. GitHub baut aber nicht automatisch eine native iPhone-App, sofern du nicht zusätzlich eine passende Build-Pipeline konfigurierst. Für den ersten Prototyp ist lokaler iOS-Export mit Xcode am einfachsten.

3.4 Keine unnötige Technik am Anfang

Für den Prototyp **nicht** einbauen:

- Online-Multiplayer
- Benutzerkonten oder Backend-Server
- Cloud-Speicherung

- In-App-Käufe oder Werbung
- Live-Streaming-Integration
- echte Bankkonten oder Zahlungsdienste
- umfangreiche 3D-Welt
- komplizierte Echtzeit-KI
- komplizierte Modding-Plattform
- dutzende Städte vor dem ersten spielbaren Tag

Das Ziel ist zunächst eine gute, lesbare, datengetriebene 2D-Managementsimulation mit einer einfachen Stadtansicht und funktionierenden Menüs.

---

4. Rollen: Du, Claude Code und die Tools

4.1 Deine Rolle

Du bist Game Director und entscheidest über:

- Spielgefühl, Prioritäten und Funktionen
- was in die erste spielbare Version gehört
- ob eine Funktion Spaß macht und verständlich ist
- welche Änderungen akzeptiert werden
- ob ein Build auf dem iPhone funktioniert
- wann ein Meilenstein abgeschlossen ist
- ob neue Funktionen das Budget und die Komplexität rechtfertigen

Du musst nicht jede Codezeile verstehen. Du solltest aber wissen, welche Dateien Claude verändert, welche Tests gelaufen sind und wie du eine funktionierende Version wiederherstellst.

4.2 Claude Code soll

- zuerst die vorhandene Projektstruktur untersuchen
- vor größeren Änderungen einen kurzen Plan nennen
- kleine, überprüfbare Arbeitspakete umsetzen
- keine nicht existierenden Godot-APIs erfinden
- keine großen Dateien oder ganze Systeme ohne Bedarf neu schreiben
- bei Änderungen an Kernsystemen passende Tests hinzufügen
- nach jedem Arbeitspaket die verfügbaren Tests ausführen
- Fehler und Einschränkungen ehrlich benennen
- Änderungen zusammenfassen und die betroffenen Dateien auflisten
- bei iOS-relevanten Änderungen sagen, was in Xcode oder auf dem iPhone geprüft werden muss
- README und technische Dokumentation aktuell halten
- niemals behaupten, ein iPhone-Build sei erfolgreich, wenn er nicht wirklich gebaut und gestartet wurde

4.3 Was Claude Code nicht automatisch kann

Claude Code kann nicht aus einer Textbeschreibung heraus garantieren, dass ein komplettes Spiel fertig, fehlerfrei oder unter iOS korrekt läuft. Es braucht einen realen Testzyklus. Der Assistent kann Code schreiben, aber Gerätesignierung, Apple-Entitlements, Exportvorlagen, Xcode-Konfiguration und reale Geräteprobleme müssen überprüft werden. Du musst Ergebnisse starten und testen, statt nur Codegenerierung als Erfolg zu werten.

4.4 Arbeitspaket-Regel

Ein Arbeitspaket sollte in der Regel nur ein klar abgegrenztes Ziel umfassen. Beispiel: „Implementiere das lokale Speichern und Laden des

aktuellen Prototyp-Spielstands und füge Tests für Speichern, Laden und fehlende Datei hinzu.“ Nicht: „Baue jetzt das ganze Spiel mit allen Features.“

---

5. Einmalige Einrichtung auf dem Mac

Schritt 1 – Projektordner anlegen

Lege einen eigenen Arbeitsordner an, beispielsweise:

`~/Developer/OFF_CAMERA`

Verwende einen kurzen Pfad ohne Sonderzeichen. Bewahre Backups nicht nur im selben Ordner auf.

Schritt 2 – Godot installieren

1. Öffne die offizielle Godot-Webseite: https://godotengine.org/download
2. Lade eine stabile Godot-4-Version für macOS herunter.
3. Installiere sie wie in der offiziellen Anleitung beschrieben.
4. Starte Godot und lege zunächst ein leeres Testprojekt an.
5. Notiere die exakte Version in `README.md`.
6. Starte eine einfache Szene, bevor du das eigentliche Projekt aufbaust.

Keine Beta- oder Nightly-Version verwenden, solange es keinen konkreten Grund gibt.

Schritt 3 – Xcode installieren

1. Öffne den Mac App Store und installiere Xcode, sofern es für deine macOS-Version verfügbar ist.
2. Starte Xcode einmal und akzeptiere erforderliche Lizenz- oder Zusatzkomponenten.
3. Öffne Xcode-Einstellungen und prüfe die installierten Plattformkomponenten.
4. Öffne Terminal und prüfe:

```
xcodebuild -version
```

Wenn der Befehl fehlschlägt, ist Xcode möglicherweise nicht vollständig installiert oder die Kommandozeilen-Tools sind nicht korrekt ausgewählt. Folge dann Apples offizieller Xcode-Dokumentation.

Offizielle Anlaufstellen:

- Xcode: https://developer.apple.com/xcode/
- Apple Developer: https://developer.apple.com/

Schritt 4 – Git installieren oder prüfen

Öffne Terminal:

```
git --version
```

Wenn macOS die Installation der Command Line Tools anbietet, folge dem Systemdialog. Konfiguriere deinen Namen und deine E-Mail für Git-Commits:

```
git config --global user.name "Dein Name"
git config --global user.email "deine-github-email@exampl
```

Verwende die E-Mail, die du für GitHub nutzen möchtest, oder eine

GitHub-Noreply-Adresse, wenn du deine private E-Mail nicht in Commits veröffentlichen willst.

Schritt 5 – Claude Code installieren

Verwende die aktuelle offizielle Installationsanleitung von Anthropic: https://docs.anthropic.com/
Die Installationsschritte und verfügbaren Abrechnungsmodelle können sich ändern; deshalb keine zufälligen Installationsbefehle aus alten Tutorials kopieren.

Nach der Installation:

1. Öffne Terminal.
2. Wechsle in den Projektordner.
3. Starte Claude Code nach der aktuellen offiziellen Anleitung.
4. Prüfe, dass Claude Code Dateien im Projekt lesen und ändern kann.
5. Starte nicht sofort mit dem kompletten Master-Prompt. Richte zuerst Repository, README und leeres Godot-Projekt ein.

Kostenhinweis: Prüfe, ob deine Nutzung über ein Abo oder über API-Abrechnung läuft. Das sind unterschiedliche Kostenmodelle. Lege eine harte eigene Budgetgrenze fest und kontrolliere die tatsächlichen Ausgaben regelmäßig.

Schritt 6 – GitHub einrichten

1. Melde dich bei https://github.com/ an oder erstelle ein Konto.
2. Erstelle ein **privates** Repository mit dem Namen `off-camera`.
3. Füge nicht blind eine fremde Projektvorlage hinzu, wenn du lokal schon Dateien hast.
4. Nutze SSH oder die von GitHub empfohlene Authentifizierungsmethode.
5. Lade keine Passwörter, API-Schlüssel, Zertifikate, private Signierungsdateien oder persönliche Zugangsdaten ins

Repository hoch.

Schritt 7 – iPhone vorbereiten

- Halte iOS auf einer unterstützten Version und Xcode/Godot kompatibel.
- Verbinde das iPhone per Kabel mit dem Mac, wenn Xcode es verlangt.
- Entsperre das Gerät und bestätige Vertrauensabfragen.
- In Xcode kann es nötig sein, Developer Mode zu aktivieren; folge der auf deinem Gerät angezeigten Anleitung.
- Für den ersten Test keine sensiblen Berechtigungen anfordern.

---

6. GitHub-Repository einrichten

6.1 Empfohlene Startstruktur

```
OFF_CAMERA/
├── project.godot
├── README.md
├── CHANGELOG.md
├── .gitignore
├── docs/
│   ├── GAME_DESIGN.md
│   ├── ARCHITECTURE.md
│   ├── ROADMAP.md
│   ├── TEST_PLAN.md
│   └── DECISIONS.md
├── scenes/
│   ├── app/
```

```
│   ├── menus/
│   ├── hud/
│   ├── city/
│   ├── activities/
│   ├── streamer/
│   ├── business/
│   ├── finance/
│   └── events/
├── scripts/
│   ├── core/
│   ├── models/
│   ├── systems/
│   ├── services/
│   └── ui/
├── data/
│   ├── activities/
│   ├── locations/
│   ├── events/
│   ├── characters/
│   └── balance/
├── assets/
│   ├── fonts/
│   ├── images/
│   └── audio/
├── tests/
├── exports/
└── .gitignore
```

Das ist eine Zielstruktur, keine Pflicht, alles am ersten Tag anzulegen. Claude soll nur Ordner erstellen, wenn sie sinnvoll genutzt werden.

6.2 `.gitignore`

Godot-Projekte erzeugen temporäre Importdateien und lokale Editorinformationen. Nutze eine aktuelle Godot-geeignete `.gitignore`-Vorlage und prüfe sie. Halte Projektdateien, Quellassets, Szenen, Scripts und Dokumentation versioniert. Generierte Exportdateien, lokale

Geheimnisse und unnötige Caches gehören normalerweise nicht ins Repository.

Ein minimales Beispiel, das vor Nutzung an die aktuelle Godot-Version angepasst werden sollte:

```
.godot/
.DS_Store
*.tmp
*.log
exports/
.env
.env.*
```

Achtung: Passe die Regeln an. Ignoriere nicht versehentlich Quellassets oder Dateien, die für den Projektaufbau erforderlich sind.

6.3 Erster Commit

Wenn das Godot-Grundprojekt existiert:

```
cd ~/Developer/OFF_CAMERA
git init
git add .
git commit -m "Initial Godot project"
```

Verbinde danach das lokale Repository mit deinem privaten GitHub-Repository entsprechend der GitHub-Anleitung und pushe den ersten Commit.

6.4 Arbeitsregeln

- Vor einer größeren Änderung: `git status` prüfen.
- Nach einem funktionierenden Meilenstein: Commit erstellen.

- Keine riesigen, ungetesteten Änderungen über viele Systeme gleichzeitig.
- Ein Branch pro größerem Feature ist sinnvoll, wenn du mit Branches umgehen kannst.
- Vor riskanten Änderungen einen bekannten funktionierenden Stand sichern.
- Claude darf niemals eigenständig Geheimnisse oder Signierungsdateien committen.
- GitHub ist ein Backup des Codes, aber kein Ersatz für regelmäßige Sicherung wichtiger persönlicher Dateien.

---

7. Godot-Projekt anlegen und iOS konfigurieren

Schritt 1 – Neues Projekt

1. Godot Project Manager öffnen.
2. Neues Projekt `OFF_CAMERA` im Projektordner anlegen.
3. Renderer für die geplante 2D-Optik wählen. Für den ersten Prototyp ist eine einfache, kompatible 2D-Konfiguration wichtiger als Spezialeffekte.
4. Projekt öffnen und eine einfache Startszene erstellen.
5. Projekt starten und prüfen, dass keine Fehler im Debugger erscheinen.
6. Projekt schließen und wieder öffnen, um sicherzustellen, dass es korrekt gespeichert ist.

Schritt 2 – Mobile UI zuerst

Für die erste Version empfiehlt sich **Hochformat**, sofern die Spieltests

keine überzeugenden Gründe für Querformat ergeben. Die Benutzeroberfläche sollte mit Containern, Ankern und responsiven Layouts arbeiten und nicht aus starren Pixelkoordinaten bestehen.

Grundregeln:

- Touch-Ziele ausreichend groß gestalten.
- Wichtige Aktionen nicht direkt an den Bildschirmrand setzen.
- Systembereiche wie Notch, Statusleiste und Home-Indikator berücksichtigen.
- Texte auf einem echten iPhone lesen können.
- Scrollbare Listen für lange Finanz- und Ereignisübersichten nutzen.
- Wichtige Informationen nicht nur über Farbe vermitteln.
- Buttons haben klare Zustände: normal, gedrückt, deaktiviert und gegebenenfalls Ladezustand.
- Bei jeder Szene prüfen, ob sie auf verschiedenen iPhone-Displaygrößen korrekt angeordnet ist.

Schritt 3 – iOS-Exportvorlagen und Export

Godot benötigt passende Exportvorlagen. Installiere die Vorlagen für die verwendete Godot-Version über die offiziellen Godot-Menüs bzw. Anweisungen. Danach:

1. Export-Preset für iOS anlegen.
2. Bundle Identifier auswählen, zum Beispiel `com.deinname.offcamera` (muss eindeutig sein).
3. Exportkonfiguration dokumentieren.
4. iOS-Projekt exportieren.
5. Das erzeugte Xcode-Projekt öffnen.
6. In Xcode Team/Signing-Konfiguration prüfen.
7. iPhone als Zielgerät wählen.
8. Build starten und App auf dem Gerät ausführen.

Die genauen Menünamen und Anforderungen unterscheiden sich je nach Godot-, Xcode- und iOS-Version. Nutze bei Abweichungen die offiziellen Dokumentationen:

- Godot iOS-Export: https://docs.godotengine.org/
- Apple Xcode: https://developer.apple.com/xcode/

Meilenstein: Noch vor der komplexen Spiellogik muss eine minimale Godot-App erfolgreich auf deinem eigenen iPhone gestartet sein. Damit wird das größte Plattformrisiko früh geklärt.

---

8. Projektarchitektur

8.1 Architekturziel

Die Spielsysteme sollen klar getrennt sein. Das verhindert, dass zum Beispiel Streaming, Behördenereignisse und Standorte ihre eigene Zeitrechnung oder eigene Kontostände erfinden.

Empfohlene Verantwortlichkeiten:

- `GameState`: aktueller Spielstand, Tag, Geld, grundlegende Spielerwerte und Referenzen auf Systemdaten.
- `TimeSystem`: einzige Autorität für Spielzeit, Tageswechsel und Zeitkosten.
- `EventSystem`: verwaltet geplante und ausgelöste Spielereignisse.
- `LedgerSystem`: einzige Autorität für Geldbuchungen und Kontostandänderungen.
- `ActivitySystem`: prüft und startet Aktivitäten, berechnet Kosten und löst Ergebnisse aus.
- `SaveSystem`: speichert und lädt einen versionierten Spielstand.
- `LocationSystem`: verwaltet Orte, Freischaltungen, Verfügbarkeit und Reise-/Aktivitätsregeln.
- `StreamerSystem`: Reichweite, Content-Aktivitäten und Creator-Fortschritt.
- `BusinessSystem`: Firmen, Standorte, Personal, Kapazität,

Betriebskosten und abstrakte Risiken.
- `AuthoritySystem`: Fristen, Prüfungen und Ereignisse als fiktionale Spielmechanik.
- `UI`-Szenen: stellen Daten dar und senden Spieleraktionen an die Systeme; sie berechnen nicht eigenständig die Kernregeln.

8.2 Grundregel: UI ist nicht die Datenbank

Ein Label darf den Kontostand anzeigen, aber nicht selbst der Kontostand sein. Wenn der Spieler Geld ausgibt, ruft die UI einen passenden Systemaufruf auf. Das Finanzsystem prüft die Buchung und aktualisiert den Zustand. Die UI liest den neuen Zustand und aktualisiert die Anzeige.

8.3 Signale und Abhängigkeiten

Nutze Godot-Signale, wo es sinnvoll ist, beispielsweise:

- `time_advanced`
- `day_changed`
- `balance_changed`
- `ledger_entry_added`
- `activity_completed`
- `event_triggered`
- `save_completed`

Nicht jedes Signal muss exakt so heißen. Wichtig ist ein konsistentes, dokumentiertes Muster. Vermeide kreisförmige Abhängigkeiten, in denen Systeme sich gegenseitig unkontrolliert aufrufen.

8.4 Datengetriebene Inhalte

Aktivitäten, Orte und Ereignisse sollen nach Möglichkeit als Daten

definiert sein. Eine Aktivität kann beispielsweise folgende Felder haben:

```
{
  "id": "stream_short_session",
  "display_name": "Kurzer Stream",
  "duration_minutes": 120,
  "energy_cost": 18,
  "money_cost": 0,
  "category": "creator",
  "requires_location": "home_studio",
  "effects": {
    "reach": 5,
    "community": 2
  }
}
```

Das ist ein illustratives Schema, kein vollständiges oder bereits in Godot eingebundenes Datenformat. Claude muss die konkrete Implementierung für Godot sauber wählen, Daten validieren und Fehler bei fehlenden IDs verständlich melden.

---

9. Die nicht verhandelbaren Kernsysteme

Diese vier Regeln sind die wichtigste technische Grundlage des Spiels.

9.1 Ein einheitlicher Kalender und Zeitfluss

Alle Aktivitäten und Ereignisse verwenden dieselbe Spielzeit. Streaming, Schlaf, Reisen, Termine, Unternehmensbetrieb und Behördenfristen dürfen nicht jeweils ihre eigene Uhr haben.

Anforderungen:

- Eine kanonische Spielzeit, beispielsweise Minuten seit Beginn der Spielwelt.
- Einheitliche Umrechnung in Tag, Uhrzeit und Wochentag.
- Aktivitäten geben ihre Zeitdauer explizit an.
- Beim Zeitfortschritt werden fällige Ereignisse in definierter Reihenfolge verarbeitet.
- Tageswechsel werden genau einmal ausgelöst.
- Keine ungewollten negativen Zeitwerte oder doppelten Tageswechsel.
- Bei einem Aktivitätsfehler darf die Zeit nicht halb fortgeschritten sein, ohne dass der Zustand konsistent bleibt.
- Keine Zeitbeschleunigung und keine unkontrollierte Echtzeit-Simulation im Hintergrund.

Beispiel: Ein zweistündiger Stream startet um 18:00. Die Spielzeit wechselt auf 20:00. Währenddessen fällige Ereignisse werden anhand derselben Uhr verarbeitet. Die Energie wird reduziert und die Creator-Werte aktualisiert. Ein Termin um 19:00 kann nicht ignoriert werden; das Spiel muss eine definierte Regel anwenden, beispielsweise Konfliktwarnung oder verpassten Termin.

9.2 Ein zentrales Finanzjournal

Alle Änderungen am Geldbestand laufen über ein einziges Ledger-/ Buchungssystem.

Jede Buchung enthält mindestens:

- eindeutige ID
- Spielzeit/Datum
- Betrag mit klarer Vorzeichenkonvention
- Kategorie
- kurze Beschreibung
- Quelle oder auslösendes System
- optionale Referenz auf Aktivität, Standort oder Ereignis
- optionaler Buchungstyp, zum Beispiel Einnahme, Ausgabe,

laufende Kosten oder Strafe

Regeln:

- UI-Buttons dürfen den Kontostand nicht direkt ändern.
- Kein System darf Geld „heimlich“ verändern.
- Jeder relevante Geldfluss erzeugt eine Buchung.
- Der Kontostand wird entweder aus einem Startwert plus Buchungen berechnet oder über einen konsistenten, getesteten Mechanismus geführt.
- Keine doppelte Buchung bei erneutem Event-Aufruf.
- Unzureichendes Geld wird nach einer klaren Regel behandelt.
- Buchungen sind nachvollziehbar und in der Finanzansicht filterbar.
- Die Speicherdatei enthält genügend Informationen, um den Kontostand nach Laden korrekt wiederherzustellen.

9.3 Skalierbare Standorte und Länder

Neue Standorte sollen überwiegend durch neue Datensätze und Konfigurationen ergänzt werden. Kein eigener Spezialcode für jeden einzelnen Standort, solange die Regeln gleich sind.

Ein Standort kann beispielsweise haben:

- ID und Anzeigename
- Stadt und Land
- Standorttyp
- Freischaltungsstatus
- Startkosten und laufende Kosten
- Kapazität
- Personalbestand
- Produktivität
- abstrakten Risiko-/Druckwert
- Liste verfügbarer Aktivitäten
- eventuelle Voraussetzungen

Neue Länder können später Währungen, Zeitzonen, Steuerregeln oder

lokale Ereignisse erfordern. Das ist nicht Teil des ersten Prototyps. Für den Anfang bleibt das Spiel bei einer vereinfachten Spielwährung und einer Stadt, damit die Architektur getestet werden kann.

9.4 Zentrale Ereignisverarbeitung

Ereignisse sollten nicht zufällig aus beliebigen UI-Skripten ausgelöst werden. Ein zentraler Event-Mechanismus verwaltet Auslöser, Fälligkeit, Bedingungen und Folgen.

Jedes Ereignis braucht:

- eindeutige ID
- Auslöser/Bedingung
- Zeitpunkt oder Prüfzeitpunkt
- Priorität, wenn mehrere Ereignisse gleichzeitig fällig sind
- Spielertext
- mögliche Auswahloptionen
- Auswirkungen auf Zeit, Werte, Geld oder Status
- Kennzeichnung, ob es bereits verarbeitet wurde

Das System muss verhindern, dass ein Ereignis beim Laden oder beim erneuten Aktualisieren doppelt ausgeführt wird.

10. Entwicklungsphasen mit konkreten Abnahmekriterien

Wichtig: Erst die aktuelle Phase abschließen, testen und sichern. Danach zur nächsten Phase wechseln. Claude Code soll keine späteren Phasen vorziehen, nur weil sie im Master-Prompt beschrieben sind.

Phase 0 – Projektgrundlage und iPhone-Export

Ziel: Leeres Spiel läuft auf deinem iPhone.

Aufgaben:

1. Godot-Projekt erstellen.
2. Git initialisieren.
3. GitHub-Repository verbinden.
4. README, Roadmap und Architektur-Notizen erstellen.
5. Minimalen Startbildschirm bauen.
6. iOS-Exportvorlagen installieren.
7. Export nach Xcode.
8. Auf iPhone installieren und starten.

Abnahmekriterien:

- Projekt öffnet ohne Fehler.
- Startszene startet in Godot.
- iOS-Build wird erfolgreich erstellt.
- App startet auf dem iPhone.
- Ein Commit markiert den funktionierenden Ausgangsstand.

Phase 1 – Spielbares Grundgerüst

Ziel: Ein vollständiger kleiner Spielablauf mit Start, Aktivität, Zeit, Geld und Save/Load.

Aufgaben:

1. Hauptmenü mit „Neues Spiel“, „Fortsetzen“ und „Einstellungen“.
2. Neues Spiel mit definierten Startwerten.
3. Spiel-HUD mit Tag, Uhrzeit, Geld, Energie und Stimmung.
4. Zwei bis vier Aktivitäten, z. B. Streamen, Ausruhen, Essen und Freizeit.
5. Zentrales Zeit-System.

6. Zentrales Ledger.
7. Ein einfaches Ereignis.
8. Speichern/Laden.
9. Fehlermeldungen und leere Zustände.

Abnahmekriterien:

- Eine Aktivität verändert die Zeit exakt wie definiert.
- Eine Ausgabe ist im Ledger zu finden.
- Eine Einnahme ist im Ledger zu finden.
- Ein Ereignis kann nur einmal verarbeitet werden.
- Speichern, App schließen, laden: Zustand bleibt konsistent.
- Die App kann auf dem iPhone ausschließlich per Touch bedient werden.

Phase 2 – Alltag und Marbella

Ziel: Der Spieler kann zwischen Zuhause und mehreren Orten navigieren und seinen Tag planen.

Aufgaben:

1. Vereinfachte 2D-Stadtansicht mit klick-/tippbaren Orten.
2. Orte: Wohnung, Studio, Café, Fitnessstudio und Büro.
3. Ortsdetails mit Beschreibung und verfügbaren Aktivitäten.
4. Aktivitäten mit Voraussetzungen und Zeitkosten.
5. Reisezeit als einfache, nachvollziehbare Regel.
6. Tageswechsel und Energie-Regeneration.
7. Termine und Konflikte.

Abnahmekriterien:

- Jeder Ort öffnet die richtige Detailansicht.
- Nicht verfügbare Aktivitäten sind verständlich deaktiviert oder erklärt.
- Reisen und Aktivitäten verwenden dasselbe Zeit-System.
- Keine Ortsansicht verändert Werte ohne Systemaufruf.

Phase 3 – Streamer-Karriere

Ziel: Die Creator-Karriere ist ein echter Management-Loop.

Aufgaben:

1. Kanalprofil und Karriereübersicht.
2. Reichweite, Community-Stimmung und Reputation als definierte Werte.
3. Content-Formate mit unterschiedlicher Dauer und Wirkung.
4. Einnahmen aus fiktionalen, abstrahierten Quellen.
5. Equipment-Upgrades mit Kosten und Effekten.
6. Zufällige, aber regelbasierte Karriereereignisse.
7. Finanzjournal für alle Geldbewegungen.

Abnahmekriterien:

- Jede Aktivität hat eine klare Dauer und definierte Auswirkungen.
- Reichweite wächst nicht unbegrenzt ohne Regeln.
- Einnahmen sind im Ledger dokumentiert.
- Wiederholtes Laden verursacht keine doppelte Auszahlung.
- UI zeigt nachvollziehbar, warum Werte gestiegen oder gefallen sind.

Phase 4 – Fiktionales Unternehmen und Standortverwaltung

Ziel: Eine kleine Organisation mit Personal, Kosten und einem zweiten Standort verwalten.

Aufgaben:

1. Unternehmens-Dashboard.
2. Personal als abstrahierte Kapazität/Produktivität.
3. Betriebskosten und Investitionen.
4. Standortfreischaltung und Ausbau.
5. Laufende Kosten beim definierten Abrechnungszeitpunkt.

6. Risiko-/Druckwert als fiktionale Spielvariable.
7. Ereignisse, die Entscheidungen erfordern.

Abnahmekriterien:

- Standortdaten liegen nicht als verstreute Spezialfälle in UI-Skripten.
- Ein zweiter Standort lässt sich durch Daten ergänzen und testen.
- Investition und laufende Kosten erscheinen im Ledger.
- Kein Standort kann Kosten mehrfach im selben Abrechnungszeitraum erzeugen.
- Kapazität und Produktivität haben verständliche Grenzen.

Phase 5 – Finanzen, Behörden und Konsequenzen

Ziel: Geld, Dokumentation, Fristen und fiktionale Prüfungen erzeugen sinnvolle Entscheidungen.

Aufgaben:

1. Finanzübersicht mit Einnahmen/Ausgaben nach Kategorie.
2. Monats-/Wochenübersicht, sofern zur Spielzeit passend.
3. Fristen und Ereigniswarteschlange.
4. Fiktionale Prüfungen und Nachfragen.
5. Auswahloptionen wie Unterlagen vorbereiten, Beratung einholen, kooperieren oder eine Konsequenz akzeptieren.
6. Kosten, Reputation und Organisationsdruck als Auswirkungen.

Abnahmekriterien:

- Behördenereignisse sind regelbasiert und nachvollziehbar.
- Fristen verwenden die zentrale Zeit.
- Entscheidungen zeigen ihre möglichen Auswirkungen, soweit das Spiel sie kennt.
- Ereignisse werden nicht doppelt ausgelöst.
- Die Simulation bleibt abstrahiert und vermittelt keine realweltlichen Anleitungen zum Verbergen illegaler Aktivitäten oder zur

Umgehung von Ermittlungen.

Phase 6 – Speichern, Laden und Datenmigration stabilisieren

Diese Phase beginnt schon in Phase 1 und wird hier gründlich abgeschlossen.

Aufgaben:

1. Save-Datei mit Versionsnummer.
2. Validierung geladener Daten.
3. Standardwerte bei optionalen fehlenden Feldern.
4. Verständliche Meldung bei beschädigtem Spielstand.
5. Backup-/Alternativspeicher, sofern sinnvoll.
6. Sicherstellen, dass Ereignisse, Buchungen und IDs eindeutig bleiben.

Abnahmekriterien:

- Ein Spielstand lässt sich mehrfach speichern und laden.
- Fehlende Datei wird sauber behandelt.
- Ungültige Datei stürzt die App nicht kommentarlos ab.
- Alte Save-Versionen werden entweder sicher migriert oder mit einer klaren Meldung abgelehnt.

Phase 7 – Mobile Politur und Performance

Ziel: Das Spiel fühlt sich auf dem iPhone brauchbar an.

Aufgaben:

1. Touch-Ziele, Schriftgrößen und Kontrast prüfen.
2. Notch-/Safe-Area-Probleme beseitigen.
3. Scrollen und Zurück-Navigation testen.
4. Keine winzigen Tabellen; lieber Karten/Listen und Filter.

5. Ladezustände, leere Zustände und Fehlertexte gestalten.
6. App-Lebenszyklus testen: in den Hintergrund, zurück, beenden, neu starten.
7. Speicher- und Performanceprobleme prüfen.
8. Akku- und Temperaturverhalten bei längeren Sessions beobachten.

Abnahmekriterien:

- Keine zentralen Buttons außerhalb des sichtbaren Bereichs.
- Kernablauf funktioniert ohne Tastatur und Maus.
- Kein unbeabsichtigter Zeitfortschritt, während die App im Hintergrund liegt.
- Kein Datenverlust nach normalem Schließen und erneutem Öffnen.

Phase 8 – Balancing und mehr Inhalte

Erst wenn der Kern stabil ist:

- weitere Orte und Aktivitäten
- mehr Figuren und Beziehungen
- weitere Content-Formate
- mehr Standorte
- Ereignisketten
- zusätzliche Finanzberichte
- weitere Städte/Länder
- visuelle Verbesserungen und Audio

Regel: Neue Inhalte dürfen die vier Kernsysteme nicht umgehen.

---

11. iPhone-Build, Installation und Tests

11.1 Der wiederholbare Build-Ablauf

1. Projekt in Godot öffnen.
2. Projekt in Godot starten und grundlegende Fehler prüfen.
3. Export-Preset für iOS prüfen.
4. iOS-Projekt exportieren.
5. Exportiertes Xcode-Projekt öffnen.
6. iPhone auswählen und Signing-Konfiguration prüfen.
7. App bauen und starten.
8. Den Kernablauf auf dem echten Gerät testen.
9. Bei Fehlern die genaue Meldung sichern und erst dann gezielt beheben.
10. Nach erfolgreichem Test einen Git-Commit erstellen.

11.2 Minimale Gerätetests pro Meilenstein

- App startet frisch.
- Neues Spiel startet.
- Buttons reagieren zuverlässig auf Touch.
- Zurück-Navigation funktioniert.
- Text ist lesbar und nicht abgeschnitten.
- Eine Aktivität kann ausgeführt werden.
- Zeit wird genau einmal aktualisiert.
- Finanzbuchung wird genau einmal erstellt.
- Speichern funktioniert.
- App kann in den Hintergrund wechseln und wieder aufgenommen werden.
- Nach vollständigem Beenden lässt sich der Spielstand laden.
- App funktioniert ohne Internet.

11.3 Gerätedaten

Die genaue iPhone-Modell- und iOS-Version dokumentieren, auf der

getestet wird. Wenn später weitere Geräte zur Verfügung stehen, mindestens eine kleinere und eine größere Displaygröße testen. Nicht behaupten, alle iPhones seien kompatibel, wenn nur ein Gerät getestet wurde.

---

12. Speicherstände, Fehlerbehebung und Qualität

12.1 Was gespeichert werden muss

Mindestens:

- Save-Version
- Spielzeit und Datum
- grundlegende Spielerwerte
- Kontostand oder Startbestand plus Ledger
- Finanzbuchungen
- Karrierewerte
- Unternehmens- und Standortdaten
- freigeschaltete Orte
- geplante Ereignisse und bereits verarbeitete Ereignis-IDs
- laufende Fristen
- wichtige Beziehungen und Fortschrittswerte
- Einstellungen

12.2 Speichern nicht nur beim App-Schließen

iOS kann eine App jederzeit unterbrechen oder beenden. Verlasse dich

nicht darauf, dass ein Exit-Callback immer ausgeführt wird. Speichere nach sinnvollen Meilensteinen, beispielsweise nach Aktivitätsabschluss oder Tageswechsel, und biete zusätzlich manuelles Speichern an, sofern es ins Design passt. Vermeide dabei übermäßige Schreibvorgänge, und stelle sicher, dass ein Speichervorgang atomar oder anderweitig gegen halbfertige Dateien geschützt ist.

12.3 Fehlerbericht für Claude Code

Wenn etwas nicht funktioniert, sende Claude:

1. Was du getan hast.
2. Was du erwartet hast.
3. Was tatsächlich passiert ist.
4. Exakte Fehlermeldung aus Godot oder Xcode.
5. Ob der Fehler auch im Godot-Editor auftritt oder nur auf dem iPhone.
6. Letzten funktionierenden Commit, falls bekannt.

Bitte Claude, zunächst die Ursache zu analysieren und einen kleinen Fix vorzuschlagen, statt das Projekt komplett neu zu schreiben.

12.4 Debug-Menü nur in Entwicklung

Ein optionales internes Debug-Menü kann später helfen, Spieltag, Werte oder Ereignisse zu prüfen. Es darf nicht versehentlich als normaler Spielinhalt erscheinen. Keine Cheats oder Testwerte dürfen unbemerkt in einen regulären Spielstand übernommen werden. Debug-Funktionen klar kennzeichnen und vor einem Release-Build kontrollieren.

13. Kosten- und Tokenkontrolle

Das Budget von **100 US-Dollar für KI-/Modellnutzung** ist eine Obergrenze, keine Garantie, dass das komplette Spiel damit fertig wird. Ein kleines spielbares Grundspiel ist realistisch als Ziel; ein großes, poliertes Managementspiel mit vielen Inhalten ist deutlich umfangreicher.

13.1 Budgetregeln

- Prüfe vor Arbeitsbeginn, wie Claude Code abgerechnet wird.
- Verwende nicht gleichzeitig mehrere kostenpflichtige KI-Dienste, ohne den Nutzen zu kennen.
- Setze, falls angeboten, ein Nutzungslimit oder Budgetwarnungen in deinem Abrechnungskonto.
- Prüfe die tatsächlichen Kosten nach jeder längeren Session.
- Lass Claude nicht wiederholt ganze Dateien oder große Projektzusammenfassungen neu ausgeben, wenn nur ein kleiner Fix nötig ist.
- Halte `README.md`, `ROADMAP.md` und `ARCHITECTURE.md` aktuell, damit du nicht bei jeder Session alles neu erklären musst.
- Bitte Claude um kurze Zusammenfassungen und konkrete Änderungen statt endloser Planung.
- Arbeite in kleinen Tickets und schließe ein Ticket ab, bevor du das nächste beginnst.
- Wenn du 70 US-Dollar erreicht hast, überprüfe Umfang und Verbrauch, bevor du fortfährst.
- Bei 90 US-Dollar nur noch notwendige Fehlerbehebung und Kernfunktionen; 100 US-Dollar ist Stoppgrenze.

13.2 Sinnvolle Priorisierung

1. iOS-App startet.
2. Spielbarer Kernloop.
3. Zeit und Ledger korrekt.
4. Speichern/Laden.
5. iPhone-UI.

6. Streamer-System.
7. Standortmanagement.
8. Behördenereignisse.
9. Mehr Inhalte und Politur.

Kein Budget für umfangreiche Kunstassets ausgeben, bevor der Spielablauf Spaß macht. Für den Anfang reichen einfache Formen, Platzhaltergrafiken und eine klare Typografie.

---

14. Arbeitsablauf mit Claude Code

14.1 Die richtige Reihenfolge jeder Session

1. Öffne Terminal im Repository.
2. Prüfe `git status`.
3. Starte Claude Code.
4. Bitte Claude zuerst, den aktuellen Stand und die relevanten Dateien zu prüfen.
5. Gib genau ein Arbeitspaket.
6. Lass Claude einen Plan mit Dateien und Risiken nennen.
7. Lass es implementieren.
8. Tests und Projektstart ausführen.
9. Änderungen selbst prüfen.
10. In Godot starten und auf iPhone testen, wenn relevant.
11. Dokumentation aktualisieren lassen.
12. Bei Erfolg committen.

14.2 Gute Arbeitsaufträge

Gut:

> Prüfe das bestehende Projekt. Implementiere ausschließlich das zentrale TimeSystem mit einer kanonischen Zeit in Minuten, Tageswechsel und Tests für Grenzfälle. Verändere noch keine Streamer- oder Business-Logik. Führe verfügbare Tests aus und nenne geänderte Dateien sowie verbleibende Risiken.

Schlecht:

> Bau jetzt das ganze Spiel fertig, mit allem, und mach es perfekt für iPhone.

Der zweite Auftrag ist zu breit und führt leicht zu unstimmiger Architektur, ungetestetem Code und hohen Kosten.

14.3 Wenn Claude große Änderungen vorschlägt

Frage:

- Warum ist diese Änderung erforderlich?
- Welche Dateien werden verändert?
- Wie wird sie getestet?
- Kann sie in kleinere Schritte zerlegt werden?
- Welche vorhandenen Funktionen könnten kaputtgehen?
- Wie stellen wir den letzten funktionierenden Stand wieder her?

14.4 Definition eines abgeschlossenen Tickets

Ein Ticket ist erst abgeschlossen, wenn:

- die Änderung implementiert ist
- das Projekt startet

- relevante Tests laufen und Ergebnisse dokumentiert sind
- keine neuen unbehandelten Fehler vorliegen
- iOS-spezifische Auswirkungen geprüft sind
- README oder technische Dokumentation bei Bedarf aktualisiert ist
- ein nachvollziehbarer Commit erstellt wurde

---

15. MASTER-PROMPT FÜR CLAUDE CODE

Anwendung: Kopiere den folgenden Prompt in Claude Code, nachdem das Repository angelegt wurde und ein leeres Godot-Projekt darin liegt. Er definiert die langfristigen Leitplanken. Er ist kein Auftrag, das komplette Spiel in einem einzigen Schritt zu generieren. Claude soll zuerst den Ist-Zustand untersuchen und dann ausschließlich Phase 0 bzw. das von dir ausdrücklich genannte Arbeitspaket bearbeiten.

MASTER-PROMPT – OFF CAMERA

Du bist mein leitender Godot-Entwickler, technischer Architekt, Testingenieur und pragmatischer Mobile-Game-Entwickler für das Projekt **OFF CAMERA**. Ich bin Game Director. Ich entscheide über Spielgefühl, Umfang und Prioritäten. Du hilfst mir, das Spiel in kleinen, nachvollziehbaren, getesteten Schritten zu bauen.

1. Projektziel

Entwickle eine Singleplayer-Lebens- und Managementsimulation mit dem Arbeitstitel OFF CAMERA. Das Spiel soll nativ auf meinem eigenen iPhone laufen. Es ist keine Android-Version geplant und eine

Veröffentlichung im App Store ist nicht erforderlich. Der Code soll in einem privaten GitHub-Repository verwaltet werden. Ich entwickle auf einem Mac und kann Xcode nutzen.

Die Spielwelt dreht sich um eine fiktionale Person, die in Marbella lebt, eine Streamer-/Creator-Karriere entwickelt und parallel eine fiktionale, kriminell geprägte Organisation verwaltet. Das Spiel umfasst Alltag, Termine, Energie, Beziehungen, Streaming, Einnahmen, Personal, Unternehmen, mehrere Standorte, Expansion, Finanzen, Behördenereignisse und Konsequenzen. Der Spieler navigiert über eine einfache Stadtkarte und klar strukturierte Management-Menüs.

Die Welt, Figuren und Organisationen sind fiktional. Keine realen Personen oder Unternehmen als tatsächliche Straftäter darstellen. Illegale Aktivitäten bleiben abstrahierte Spielmechaniken; implementiere keine realweltlich anwendbaren Anleitungen zu Straftaten, Geldwäsche, Steuerhinterziehung oder Ermittlungsumgehung. Das Gameplay soll Entscheidungen, Ressourcen, Risiken und Konsequenzen modellieren, nicht reale kriminelle Methoden erklären.

2. Plattform und Technik – nicht verhandelbar

- Zielplattform: iOS auf iPhone.
- Engine: stabile Godot-4-Version, genaue Versionsnummer dokumentieren.
- Sprache: GDScript, sofern kein zwingender technischer Grund dagegen spricht.
- UI und Eingabe: Mobile-first, Touch-first, responsive Layouts.
- Build: Godot iOS Export + Xcode auf macOS.
- Repository: Git/GitHub.
- Offline-Spielbarkeit ist für den Kern verpflichtend.
- Keine Android-spezifischen Aufgaben, es sei denn, ich ändere die Vorgabe ausdrücklich.
- Kein App-Store-Release-Workflow als Projektziel. Trotzdem muss der iOS-Build korrekt signiert und auf einem eigenen Gerät installierbar sein.
- Keine verpflichtenden Onlinekonten, Server oder Cloud-Speicher im ersten Spiel.
- Keine Werbung, In-App-Käufe, Multiplayer- oder Streaming-API-

Integration im ersten Spiel.
- Kein 3D-Zwang. Bevorzuge eine übersichtliche, performante 2D-Managementoberfläche.
- Bevorzuge robuste, einfache Lösungen vor komplizierten Frameworks.

Wenn eine Anforderung von der installierten Godot-/Xcode-Version abhängt, prüfe die vorhandene Projektkonfiguration und die verfügbare Dokumentation. Erfinde keine API, Exportoption oder Editorfunktion. Wenn du etwas nicht verifizieren kannst, kennzeichne die Unsicherheit und schlage einen konkreten Test vor.

3. Mobile-First-Qualitätsanforderungen

Die App muss auf einem echten iPhone gut bedienbar sein. Plane insbesondere:

- Hochformat als Startannahme, sofern keine spätere Designentscheidung dagegenspricht.
- Responsive Layouts statt fester Positionierung für nur eine Auflösung.
- Ausreichend große Touch-Ziele.
- Lesbare Schrift und ausreichenden Kontrast.
- Safe Areas für Notch, Statusleiste und Home-Indikator.
- Scrollbare Listen für längere Informationen.
- Keine Kernaktion darf Maus-Hover voraussetzen.
- Keine Tastaturkürzel als einzige Bedienungsmöglichkeit.
- Klare Zurück-Navigation und verständliche Zustände für Laden, Fehler und leere Listen.
- Keine unnötigen Berechtigungen. Der Prototyp benötigt keine Kamera, kein Mikrofon, keinen Standort und keine Kontakte.
- Die App darf nicht ungewollt Spielzeit fortschreiben, während sie im Hintergrund liegt.
- Speichern darf nicht nur beim Schließen stattfinden, da iOS Apps unterbrechen oder beenden kann.
- Offline-Nutzung für den Kernloop.
- Teste auf dem Simulator und, sobald möglich, auf einem echten iPhone.

Verwende Godot-Container, Anker und responsive UI-Praktiken. Vermeide eine Oberfläche, die nur auf dem Entwicklungs-Mac gut aussieht. Wenn die tatsächliche iPhone-Auflösung nicht bekannt ist, verwende eine dokumentierte, responsive Basis und fordere mich zu einem Gerätetest auf.

4. Wichtigste Designprinzipien

- Zuerst ein kleiner, wirklich spielbarer Prototyp.
- Jede Phase muss einen lauffähigen Zustand erhalten.
- Baue nur das, was das aktuelle Arbeitspaket benötigt.
- Keine riesigen Code-Dumps oder unnötigen Abhängigkeiten.
- Keine unnötigen Neuimplementierungen bereits funktionierender Systeme.
- Kernregeln gehören in Systems/Services/Models, nicht in UI-Skripte.
- Inhalte wie Orte, Aktivitäten und Ereignisse möglichst datengetrieben definieren.
- Verwende eindeutige IDs statt Anzeigenamen als Schlüssel.
- Alle Zufallsmechaniken müssen kontrollierbar und testbar sein; wo nötig, verwende einen Seed.
- Behandle Fehler ausdrücklich und verständlich.
- Dokumentiere technische Entscheidungen in `docs/DECISIONS.md`.
- Aktualisiere README und Roadmap, wenn sich der reale Projektstand ändert.

5. Vier Architekturregeln, die niemals umgangen werden dürfen

5.1 Ein zentrales Zeit-System

Es gibt genau eine kanonische Spielzeit. Alle Aktivitäten, Termine, Reisen, Tageswechsel, Unternehmensabrechnungen, Ereignisse und Behördenfristen greifen auf dasselbe System zu. Kein Teil der UI und kein Fachsystem führt eine eigene Uhr.

Speichere Zeit intern in einer robusten kanonischen Einheit, zum Beispiel Minuten seit Spielbeginn, und wandle sie zentral in Datum/Uhrzeit um. Definiere, wann ein Tag beginnt und endet. Verarbeite beim Zeitfortschritt alle fälligen Ereignisse in einer klar dokumentierten Reihenfolge. Verhindere doppelte Tageswechsel, doppelte Ereignisse, negative Zeit und inkonsistente Zwischenzustände.

Es gibt keine unkontrollierte Echtzeit-Simulation und keine Zeitbeschleunigung. Zeit vergeht durch definierte Aktionen und Ereignisse. Die Zeit darf nur über das zentrale System verändert werden.

5.2 Ein zentrales Finanzjournal / Ledger

Jede relevante Geldbewegung muss durch genau ein zentrales Finanzsystem laufen. UI-Skripte und Fachsysteme dürfen den Kontostand nicht direkt verändern. Jede Buchung hat eindeutige ID, Zeit, Betrag, Kategorie, Beschreibung und Quelle/Referenz. Verhindere doppelte Buchungen, insbesondere wenn Ereignisse erneut verarbeitet oder Spielstände geladen werden.

Das Ledger muss Einträge sortieren, filtern und anzeigen können. Kontostand und Buchungshistorie müssen nach Speichern und Laden konsistent bleiben. Jede Investition, Aktivitätsausgabe, Creator-Einnahme, laufende Kostenposition und fiktionale Strafe muss nachvollziehbar gebucht werden. Definiere eine einheitliche Vorzeichenkonvention und teste sie.

5.3 Skalierbare Standorte und Länder

Orte, Standorte, Unternehmen und Länder sollen datengetrieben modelliert werden. Ein neuer Standort desselben Typs soll möglichst durch Konfiguration und Daten ergänzt werden können, nicht durch eine neue Sonderlogik in jedem System. Halte Standortdaten, Kosten, Kapazitäten, Freischaltungen und lokale Ereignisse getrennt von der UI.

Für den ersten Prototyp implementiere nur Marbella und wenige Orte. Architektur für spätere Länder vorbereiten, aber keine komplizierte Mehrwährungs- oder internationale Steuersimulation vorzeitig bauen.

Erweitere nur, wenn ein Meilenstein es erfordert.

5.4 Zentrales Ereignis-System

Ereignisse haben eindeutige IDs, Bedingungen, Fälligkeit, Priorität, Text, Auswahlmöglichkeiten, Folgen und Verarbeitungsstatus. Die Event-Verarbeitung muss deterministisch genug sein, um reproduzierbar getestet zu werden. Ein Ereignis darf beim Laden nicht unbeabsichtigt erneut ausgeführt werden. Zeit-, Geld- und Wertänderungen aus Ereignissen müssen durch die zuständigen Systeme laufen.

6. Gewünschte Spielsysteme

6.1 Alltag

Der Spieler hat einen Kalender, Energie, Stimmung und verfügbare Zeit. Er kann schlafen, essen, trainieren, Freizeit verbringen, soziale Termine wahrnehmen und Orte aufsuchen. Werte sind abstrakte Spielwerte, keine medizinische Simulation. Aktivitäten haben Dauer, Voraussetzungen, Kosten und Auswirkungen.

6.2 Stadt und Orte

Baue eine einfache, gut lesbare Stadtansicht. Starte mit Wohnung, Studio, Café, Fitnessstudio und Büro. Jeder Ort hat ID, Name, Beschreibung, Freischaltungsstatus und verfügbare Aktivitäten. Orte müssen mit dem zentralen Zeit-System interagieren. Für den Prototyp darf die Karte stilisiert und einfach sein; sie muss keine offene 3D-Welt sein.

6.3 Streamer-/Creator-Karriere

Implementiere einen nachvollziehbaren Loop aus Content-Auswahl, Zeitkosten, Energieverbrauch, Reichweite/Community-Effekten, Einnahmen und Ereignissen. Reichweite, Community, Reputation und Geld müssen definierte Grenzen oder Wachstumsregeln besitzen.

Fiktionale Einnahmen aus Kooperationen, Werbung oder Mitgliedschaften werden über das Ledger verbucht. Keine reale Streaming-Plattform-Integration im ersten Spiel.

6.4 Fiktionales Unternehmensmanagement

Unternehmen haben Standorte, Personal, Kapazität, Produktivität, laufende Kosten und abstrakte Risikowerte. Managementaktionen haben Zeit- und Geldkosten. Die Eröffnung eines Standorts muss ein sinnvoller Meilenstein sein. Laufende Kosten dürfen pro Abrechnungsperiode nur einmal anfallen. Alle Auswirkungen werden über zentrale Systeme verarbeitet.

6.5 Finanzen und Behördenereignisse

Zeige Einnahmen, Ausgaben, Kategorien, Zeitraum und Buchungshistorie. Behördenereignisse sind fiktionale, abstrakte Entscheidungsereignisse mit Fristen und Konsequenzen. Optionen können Dokumentation vorbereiten, Beratung einholen, kooperieren, Entscheidungen anfechten oder Folgen akzeptieren. Keine Anleitungen zur Steuerhinterziehung, Geldwäsche, Verschleierung oder Ermittlungsumgehung implementieren. Ereignisse sollen spielerisch verständlich und fair sein.

6.6 Beziehungen und Ereignisse

Beziehungen können als einfache Fortschrittswerte starten. Ereignisse dürfen Optionen und Folgen haben. Vermeide eine riesige Dialog-Engine, bis der Grundloop stabil ist.

7. Save-/Load-System

Baue früh ein lokales Save-System. Der Save enthält eine Schema-/Versionsnummer. Speichere alle relevanten Zustände, einschließlich zentraler Zeit, Ledger, geplanter Ereignisse, verarbeiteter Ereignis-IDs, Spielerwerte, Karriere, Standorte, Freischaltungen und Einstellungen.

- Behandle fehlende Save-Dateien als normalen Zustand.
- Behandle beschädigte/ungültige Dateien mit verständlicher Fehlermeldung, nicht mit stillem Absturz.
- Validiere Daten beim Laden.
- Nutze sinnvolle Standardwerte für optionale neue Felder.
- Vermeide doppelte Buchungen oder Events nach dem Laden.
- Speichere nach sinnvollen Meilensteinen und biete manuelles Speichern, falls es ins Design passt.
- Verlasse dich nicht nur auf App-Schließereignisse.
- Dokumentiere, wie spätere Save-Versionen migriert werden oder warum eine Version nicht unterstützt wird.

8. Projektstruktur und Wartbarkeit

Erkunde zuerst die bestehende Struktur. Verwende oder schlage eine klare Aufteilung vor, beispielsweise:

- `scenes/` für Godot-Szenen
- `scripts/core/` für Spielzustand und zentrale Steuerung
- `scripts/systems/` für Zeit, Events, Ledger, Aktivitäten und Save/Load
- `scripts/models/` für Datenmodelle
- `scripts/services/` für unterstützende Logik
- `scripts/ui/` für UI-Logik
- `data/` für datengetriebene Aktivitäten, Orte und Ereignisse
- `assets/` für Quellgrafiken, Fonts und Audio
- `tests/` für Tests
- `docs/` für Architektur, Design, Roadmap und Testplan

Die Struktur ist ein Vorschlag, kein Grund, vorhandene Dateien sinnlos zu verschieben. Halte Scripts klein genug, um sie zu verstehen. Vermeide globale Autoloads für jedes beliebige System. Nutze Autoloads nur für zentrale Dienste, deren Lebenszyklus tatsächlich global sein muss.

9. Tests und Qualitätskontrolle

Für jedes Kernsystem müssen Tests oder nachvollziehbare Testfälle

existieren. Teste mindestens:

- Zeitfortschritt und Tageswechsel
- Aktivitätsdauer und Zeitkosten
- mehrere gleichzeitig fällige Ereignisse
- Ereignis wird nicht doppelt verarbeitet
- Ledger-Einnahme und -Ausgabe
- Kontostand bei unzureichendem Geld
- keine doppelte Buchung nach erneutem Event-Aufruf
- Save/Load-Rundlauf
- fehlende oder ungültige Save-Datei
- Standortkosten pro Abrechnungszeitraum nur einmal
- UI-Zustände ohne Daten
- Touch-Navigation auf iPhone

Führe die Tests aus, die in der vorhandenen Umgebung tatsächlich möglich sind. Unterscheide klar zwischen automatisierten Tests, manuellem Editor-Test und echtem iPhone-Test. Behaupte nie, einen Test ausgeführt zu haben, wenn er nicht ausgeführt wurde. Wenn ein iPhone-Test nicht möglich ist, nenne die konkrete manuelle Prüfung, die ich durchführen muss.

10. Arbeitsmodus – sehr wichtig

Arbeite in kleinen, begrenzten Arbeitspaketen. Starte nicht damit, das komplette Spiel in einem Durchlauf zu generieren.

Für jedes Arbeitspaket:

1. Prüfe `git status` und die relevanten Dateien.
2. Fasse den Ist-Zustand kurz zusammen.
3. Schlage einen kleinen Plan vor.
4. Nenne Dateien, die du ändern willst.
5. Warte auf meine Zustimmung, wenn die Änderung groß, riskant oder außerhalb der aktuellen Phase ist.
6. Implementiere nur das vereinbarte Ziel.
7. Führe passende Tests oder Projektchecks aus.
8. Prüfe mögliche Seiteneffekte.
9. Liste geänderte Dateien und wichtige Entscheidungen auf.

10. Nenne Testergebnisse ehrlich und konkret.
11. Gib mir die exakten Schritte für Godot/Xcode/iPhone, wenn manuell getestet werden muss.
12. Aktualisiere Dokumentation, wenn nötig.
13. Empfiehl einen Commit, aber committe nur, wenn ich es erlaube oder wir eine klare Commit-Regel vereinbart haben.

Wenn ein Test scheitert, verberge das nicht. Analysiere zuerst die Ursache. Führe keine große Neuschreibung durch, um einen kleinen Fehler zu beheben.

11. Kostenkontrolle

Das maximale Budget für KI-/Modellnutzung beträgt 100 US-Dollar. Es ist eine harte Grenze. Du kannst nicht zuverlässig selbst meine tatsächliche Kontobelastung kennen; fordere mich deshalb regelmäßig auf, das Nutzungsdashboard zu prüfen. Halte Ausgaben niedrig:

- keine unnötigen Wiederholungen des kompletten Master-Prompts
- keine großen Dateien vollständig ausgeben, wenn ein Diff genügt
- keine unnötigen Dependencies
- keine großflächigen Refactorings ohne Not
- kurze Zusammenfassungen und kleine Arbeitspakete
- bei jedem Meilenstein Aufwand und nächste Priorität nennen

Wenn ich melde, dass 70 US-Dollar erreicht sind, schlage eine Budgetprüfung vor. Bei 90 US-Dollar priorisieren wir nur Kernfunktionen und Fehlerbehebung. Bei 100 US-Dollar keine weitere kostenpflichtige KI-Arbeit ohne meine bewusste neue Entscheidung.

12. Reihenfolge der Entwicklungsphasen

Arbeite in dieser Reihenfolge, sofern ich sie nicht ausdrücklich ändere:

Phase 0: Godot-Projekt, Git/GitHub, minimaler Startbildschirm, iOS-Export und erfolgreicher Start auf meinem iPhone.
Phase 1: Startmenü, neues Spiel, HUD, wenige Aktivitäten, zentrales Zeit-System, Ledger, ein Ereignis, Save/Load.

Phase 2: Alltag, Marbella-Karte, Orte, Reisezeit und Termine.
Phase 3: Streamer-/Creator-Karriere mit nachvollziehbaren Effekten und Einnahmen.
Phase 4: Fiktionales Unternehmensmanagement, Personal und zweiter Standort.
Phase 5: Finanzen, Fristen, Behördenereignisse und Konsequenzen.
Phase 6: Save/Load-Stabilisierung, Validierung und Migration.
Phase 7: iPhone-Politur, Layout, Performance und App-Lebenszyklus.
Phase 8: mehr Inhalte, weitere Standorte und später weitere Länder.

Jede Phase braucht eine Liste von Abnahmekriterien. Beginne keine nächste Phase, bevor die aktuelle mindestens auf dem vorgesehenen Testniveau funktioniert.

13. Was du jetzt als Erstes tun sollst

Führe **noch keine komplette Spielentwicklung** aus. Untersuche zunächst das Repository und die vorhandene Godot-Projektkonfiguration. Berichte:

1. Welche Dateien und Szenen existieren?
2. Welche Godot-Version ist erkennbar?
3. Ist Git korrekt initialisiert?
4. Gibt es offensichtliche Fehler oder fehlende Dateien?
5. Was ist der kleinste nächste Schritt für Phase 0?
6. Welche Schritte muss ich in Godot/Xcode selbst durchführen, um das Spiel erstmals auf meinem iPhone zu starten?
7. Welche Voraussetzungen sind noch unklar?

Wenn das Repository leer ist, schlage einen minimalen Projektaufbau vor. Erstelle nicht ohne Rücksprache eine riesige Struktur. Warte auf meine Freigabe, bevor du größere Änderungen vornimmst.

Oberstes Ziel: Ein kleiner, sauberer, nachweislich auf meinem iPhone spielbarer Kern ist wertvoller als eine große Menge ungetesteten Codes.

---

16. Folgeprompts für jede Entwicklungsphase

Nutze diese Prompts einzeln, nicht alle gleichzeitig.

Prompt A – Phase 0: iOS-Projekt zum Laufen bringen

> Arbeite ausschließlich an Phase 0 von OFF CAMERA. Untersuche zuerst Repository, Godot-Version und vorhandene Szenen. Erstelle nur das notwendige minimale Projektgerüst und einen einfachen Startbildschirm. Dokumentiere die Godot-Version, Exportvoraussetzungen und die genauen Schritte für den iOS-Export nach Xcode. Erfinde keine APIs und behaupte nicht, dass ein echter iPhone-Build getestet wurde, wenn er nicht tatsächlich gelaufen ist. Gib mir nach der Implementierung eine kurze Testcheckliste. Noch keine Gameplay-Systeme hinzufügen.

Prompt B – Phase 1: Zeit, Ledger und Spielschleife

> Implementiere ausschließlich den ersten spielbaren Kern von OFF CAMERA: neues Spiel, HUD, wenige Aktivitäten, zentrale Spielzeit, zentrales Ledger, ein Ereignis sowie lokales Save/Load. Verwende eine einzige Zeitquelle und einen einzigen Geldbuchungsweg. UI darf Zeit oder Geld nicht direkt ändern. Ergänze Tests für Aktivitätsdauer, Tageswechsel, Einnahme, Ausgabe, doppelte Ereignisse und Save/Load. Verändere keine späteren Systeme. Nenne nachher alle geänderten Dateien und die exakten Tests, die wirklich gelaufen sind.

Prompt C – Phase 2: Stadt und Alltag

> Erweitere den bestehenden spielbaren Kern um eine einfache Marbella-Stadtansicht und fünf Orte: Wohnung, Studio, Café, Fitnessstudio und Büro. Orte und verfügbare Aktivitäten sollen datengetrieben sein. Reisezeit und Aktivitätsdauer müssen dasselbe zentrale TimeSystem verwenden. Implementiere nur einfache Platzhaltergrafiken. Keine 3D-Welt und keine neuen Backend-Abhängigkeiten. Teste Navigation, Voraussetzungen, Zeitkosten und Save/Load.

Prompt D – Phase 3: Streamer-Karriere

> Ergänze einen kleinen, vollständigen Streamer-/Creator-Management-Loop. Implementiere Kanalwerte, wenige Content-Formate, Energie- und Zeitkosten, nachvollziehbare Effekte und fiktionale Einnahmen. Jede Geldbewegung muss über das bestehende Ledger laufen. Vermeide unbegrenztes Wachstum und erkläre im UI, warum Werte sich verändert haben. Ergänze Tests für Einnahmen, Wiederholungen und Save/Load. Schreibe keine reale Plattformintegration.

Prompt E – Phase 4: Unternehmensmanagement

> Ergänze ein kleines fiktionales Unternehmensmanagement mit einem Startstandort und einem freischaltbaren zweiten Standort. Standorte, Personal, Kapazität, Produktivität und laufende Kosten müssen klar modelliert sein. Neue Standorte desselben Typs sollen datengetrieben hinzufügbar sein. Alle Kosten laufen über das Ledger. Stelle sicher, dass laufende Kosten pro Abrechnungszeitraum nicht doppelt berechnet werden. Halte illegale Tätigkeiten abstrakt und nicht als reale Handlungsanleitung.

Prompt F – Phase 5: Behördenereignisse und Finanzen

> Ergänze eine Finanzübersicht mit Filtern und ein kleines, fiktionales Behörden-/Prüfungsereignis mit Frist und mehreren Spieleroptionen. Alle Fristen verwenden das zentrale TimeSystem, Geldfolgen laufen über das Ledger und Ereignisse dürfen nicht doppelt verarbeitet werden. Optionen sollen nachvollziehbare Spielkonsequenzen haben. Keine Anleitungen zu Steuerhinterziehung, Geldwäsche, Verschleierung oder Ermittlungsumgehung. Ergänze Tests für Fristablauf, Speichern/Laden und einmalige Verarbeitung.

Prompt G – iPhone-UI-Audit

> Prüfe das gesamte bisherige Spiel ausschließlich auf mobile Bedienbarkeit und iPhone-Layout. Suche nach starren Positionen, zu kleinen Touch-Zielen, abgeschnittenem Text, nicht scrollbaren Listen, problematischer Safe-Area-Nutzung und Maus-/Tastaturabhängigkeiten. Liste zuerst die Probleme nach Schweregrad. Behebe sie in kleinen Schritten, ohne die Spielregeln umzuschreiben. Sage ausdrücklich, welche Probleme nur auf einem echten iPhone zuverlässig geprüft werden können.

Prompt H – Save-/Load-Audit

> Prüfe das Save-/Load-System auf Vollständigkeit, ungültige Dateien, fehlende Felder, doppelte Events, doppelte Buchungen und Versionswechsel. Erstelle Tests und behebe nur die festgestellten Probleme. Der Spielstand muss nach Speichern, vollständigem App-Neustart und Laden denselben konsistenten Zustand wiederherstellen. Dokumentiere das Save-Schema und dessen Versionierung.

Prompt I – Fehlerbehebung

> Untersuche den folgenden Fehler, ohne das Projekt großflächig neu zu schreiben. Reproduziere ihn, falls möglich, identifiziere die Ursache, schlage einen minimalen Fix vor und ergänze einen Test, der die Regression verhindert. Fehlerbeschreibung: [HIER EINFÜGEN]. Erwartetes Verhalten: [HIER EINFÜGEN]. Tatsächliches Verhalten: [HIER EINFÜGEN]. Logs: [HIER EINFÜGEN].

Prompt J – Vorbereitung eines sicheren Meilenstein-Commits

> Prüfe den aktuellen Git-Diff. Fasse jede Änderung kurz zusammen. Suche nach versehentlichen Geheimnissen, unnötigen generierten Dateien, Debug-Code, nicht verwendeten Dateien und offensichtlichen Regressionen. Führe relevante Tests aus und nenne die Ergebnisse. Erstelle keinen Commit ohne meine Freigabe. Gib mir anschließend einen passenden Commit-Namen.

---

17. Checklisten und Definition of Done

17.1 Checkliste vor jedem neuen Feature

- ☐ Was soll der Spieler konkret tun können?
- ☐ Welche Daten werden benötigt?
- ☐ Welche zentrale Systeme werden angesprochen?
- ☐ Wird Zeit verändert? Falls ja: ausschließlich über TimeSystem.

- ☐ Wird Geld verändert? Falls ja: ausschließlich über Ledger.
- ☐ Wird ein Ereignis ausgelöst? Falls ja: eindeutige ID und Schutz gegen doppelte Verarbeitung.
- ☐ Muss der Zustand gespeichert werden?
- ☐ Ist die UI auf einem iPhone sinnvoll bedienbar?
- ☐ Welche Tests beweisen, dass die Funktion korrekt ist?
- ☐ Welche Dateien werden geändert?

17.2 Checkliste nach jedem Feature

- ☐ Projekt startet in Godot.
- ☐ Keine neuen Parser-/Scriptfehler.
- ☐ Relevante Tests ausgeführt.
- ☐ Save/Load geprüft, falls Daten betroffen sind.
- ☐ iOS-Auswirkungen geprüft.
- ☐ UI auf kleinem Bildschirm geprüft.
- ☐ Dokumentation aktualisiert.
- ☐ Git-Diff gelesen.
- ☐ Funktionierender Commit erstellt oder vorbereitet.

17.3 Release-ähnlicher Test für dich selbst

Auch ohne App Store solltest du vor einem längeren Spielstand prüfen:

- ☐ Frische Installation startet.
- ☐ Neues Spiel beginnt.
- ☐ Ein kompletter Spieltag ist spielbar.
- ☐ Mindestens eine Einnahme und eine Ausgabe sind korrekt.
- ☐ Mindestens ein Ereignis wird korrekt ausgelöst.
- ☐ Kein Ereignis wird doppelt ausgelöst.
- ☐ Save/Load übersteht vollständiges Schließen der App.
- ☐ App funktioniert offline.
- ☐ UI ist lesbar und touchfähig.
- ☐ Keine kritischen Fehlermeldungen in Godot/Xcode.
- ☐ iOS-Signierung und Geräteinstallation funktionieren.

- ☐ Der aktuelle Stand ist in Git gesichert.

17.4 Definition of Done für den ersten spielbaren Prototyp

Der erste Prototyp ist fertig, wenn du auf deinem iPhone ein neues Spiel beginnen, eine Aktivität durchführen, Zeit und Werte verändern, Geldbewegungen im Ledger nachvollziehen, ein Ereignis erleben und den Spielstand nach einem vollständigen App-Neustart wieder laden kannst. Die Stadtkarte und alle späteren Systeme sind für diesen ersten Meilenstein noch nicht zwingend.

---

18. Häufige Probleme

„Das Projekt läuft im Godot-Editor, aber nicht auf dem iPhone.“

Prüfe Exportvorlagen, iOS-Export-Preset, Bundle Identifier, Xcode-Version, Signierung/Team, ausgewähltes Gerät und die Build-Logs. Nicht sofort das Gameplay umbauen. Exportfehler sind häufig von Scriptfehlern unabhängig.

„Claude sagt, es ist fertig, aber ich sehe nichts auf dem iPhone.“

Frage nach dem genauen Testnachweis. Wurde nur Code geändert, das Projekt im Editor gestartet, ein iOS-Projekt exportiert oder tatsächlich ein

Build auf einem Gerät ausgeführt? Das sind unterschiedliche Teststufen.

„Die Buttons sind zu klein.“

Prüfe Touch-Zielgrößen, Container, Mindestgrößen, Abstände und Safe Areas. Lass Claude ein UI-Audit durchführen, statt einzelne Buttons willkürlich zu vergrößern.

„Das Geld stimmt nach dem Laden nicht.“

Untersuche, ob der Kontostand aus Buchungen rekonstruiert wird, ob beim Laden Ereignisse erneut ausgeführt werden oder ob ein System direkt den Kontostand verändert. Das ist ein Fehler im Kernsystem und muss vor neuen Features behoben werden.

„Zeit springt doppelt oder Termine werden übersprungen.“

Prüfe, ob mehrere Systeme Zeit unabhängig fortschreiben, ob Ereignisse bei Grenzwerten doppelt ausgewertet werden oder ob der Tageswechsel mehr als einmal ausgelöst wird. Es darf nur eine Zeitquelle geben.

„Claude ändert zu viele Dateien.“

Stoppe die Arbeit, sichere den letzten guten Commit und fordere eine Analyse mit kleinem Plan. Lass den Assistenten nur die notwendigen Dateien ändern. Bei größeren Refactorings erst eine Begründung und Tests verlangen.

„Die Kosten steigen schnell.“

Kleinere Arbeitspakete, kürzere Zusammenfassungen, weniger Wiederholungen, weniger Kontext und regelmäßige Git-Commits. Priorisiere spielbaren Kern und Fehlerbehebung vor zusätzlichen Inhalten. Prüfe das reale Nutzungsdashboard – Claude kann die tatsächliche Abrechnung nicht zuverlässig aus dem Code ableiten.

„Ich will sofort zehn Städte und sehr viele Systeme.“

Lege sie als spätere Inhalte fest. Zuerst muss eine Stadt mit wenigen Aktivitäten und einer nachvollziehbaren Spielschleife funktionieren. Eine skalierbare Architektur bedeutet, dass spätere Erweiterungen möglich sind – nicht, dass alle Inhalte vor dem ersten Test gebaut werden müssen.

---

19. Die ersten zwei Arbeitssitzungen

Sitzung 1 – Umgebung und erstes iPhone-Build

Ziel: Nach dieser Sitzung hast du eine minimale App auf deinem iPhone gestartet.

1. Godot installieren und Version notieren.
2. Xcode installieren/starten und Lizenz-/Komponentenabfragen

abschließen.
3. Git-Version prüfen.
4. Claude Code nach offizieller Anleitung einrichten.
5. Privates GitHub-Repository erstellen.
6. Leeres Godot-Projekt in `OFF_CAMERA` anlegen.
7. Minimale Startszene erstellen.
8. Ersten Commit erstellen.
9. iOS-Exportvorlagen installieren und Export-Preset einrichten.
10. Xcode-Projekt öffnen und Signing konfigurieren.
11. Auf dem iPhone starten.
12. Funktionierenden Stand committen und dokumentieren.

Wenn dieser Ablauf scheitert, löse zuerst den Export-/Signing-Fehler. Baue noch keine großen Gameplay-Systeme.

Sitzung 2 – Erste echte Spielschleife

Ziel: Du kannst auf dem iPhone eine Aktivität auswählen, Zeit verbrauchen und einen Effekt sehen.

1. Claude soll das bestehende Projekt untersuchen.
2. Master-Prompt als Architekturleitlinie bereitstellen.
3. Nur Phase 1 beauftragen.
4. HUD mit Tag, Uhrzeit, Geld und Energie bauen.
5. Zentrales TimeSystem implementieren.
6. Zwei Aktivitäten mit klaren Zeitkosten hinzufügen.
7. Einen kleinen Event-/Feedback-Mechanismus einbauen.
8. Automatische/gezielte Tests ausführen.
9. In Godot starten.
10. iOS exportieren und auf dem iPhone testen.
11. Bugliste erstellen und beheben.
12. Erst danach Ledger und Save/Load weiter ausbauen, falls nicht schon im aktuellen Ticket enthalten.

Wichtig: Lieber nach der zweiten Sitzung zwei wirklich funktionierende Aktivitäten als zehn halbfertige Menüs.

Schlussregel

Baue OFF CAMERA wie ein echtes Softwareprojekt: kleine Änderungen, Tests, Git-Commits, echte iPhone-Tests und klare Abnahmekriterien. Die Reihenfolge ist bewusst gewählt: **zuerst iOS-Build, dann Spielschleife, danach Zeit und Finanzen stabilisieren, anschließend Inhalte und Komplexität ergänzen.**

Die Qualität des Spiels wird nicht daran gemessen, wie viel Code Claude in einer Session erzeugt, sondern daran, ob du auf deinem iPhone zuverlässig spielen kannst und die Systeme über längere Spielstände hinweg konsistent bleiben.